

This is CS50

Week 4

Kelly Ding (she/her)

Preceptor

kelly@cs50.harvard.edu

- What are **pointers**, and how can we become familiar with their **syntax**?
- What is **dynamic memory**, and how should we use it?
- How can we **read** and **write** data from a file?

Part 0

Hexadecimal Notation

Quick Hexadecimal Review!

- For this week's pset, it's important to understand hexadecimal notation!
 - Hexadecimal numbers are indicated by a "0x" in the front.
 - There are 16 digits: 0-9, A (10), B (11), C (12), D (13), E (14), F (15)
 - Hex notation is typically used for representing addresses in memory.
- Here's how to interpret a hex number:

0x _____ _____ _____ _____
 x 16³ x 16² x 16¹ x 16⁰

Part 1

Pointers

?	?	?	?	?	?	?	?
0x08	0x09	0x0A	0x0B	0x0C	0x0D	0x0E	0x0F
?	?	?	?	?	?	?	?
0x10	0x11	0x12	0x13	0x14	0x15	0x16	0x17
?	?	?	?	?	?	?	?
0x18	0x19	0x1A	0x1B	0x1C	0x1D	0x1E	0x1F

```
int n = 4;
```

n 4 0x08				? 0x0C	? 0x0D	? 0x0E	? 0x0F
? 0x10	? 0x11	? 0x12	? 0x13	? 0x14	? 0x15	? 0x16	? 0x17
? 0x18	? 0x19	? 0x1A	? 0x1B	? 0x1C	? 0x1D	? 0x1E	? 0x1F

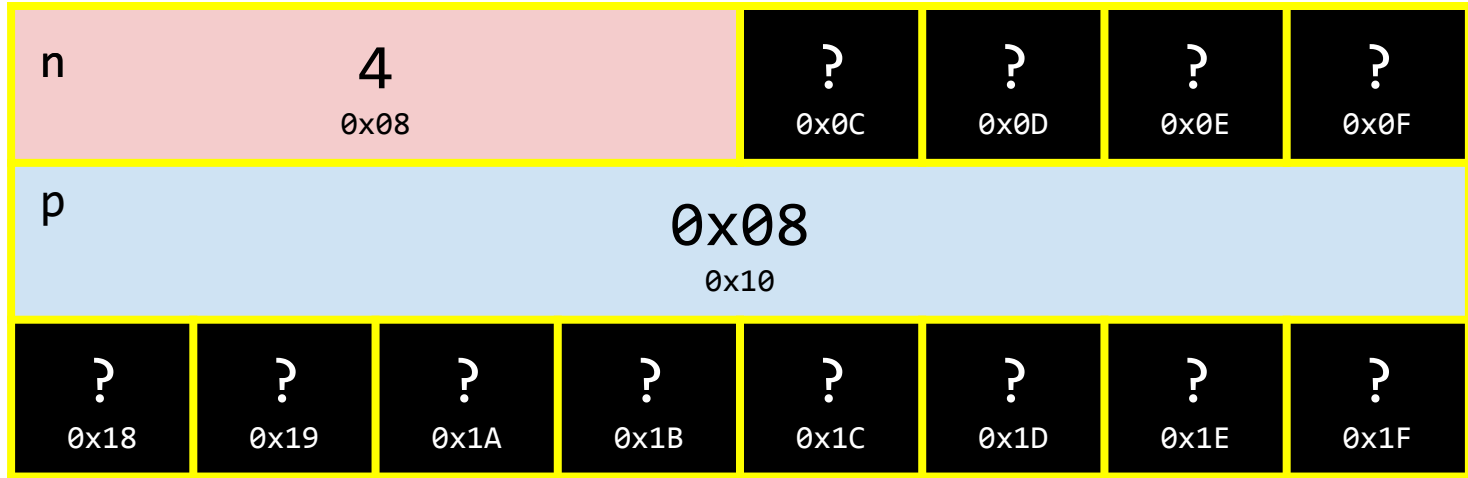
What's the address of n?

n 4 0x08				? 0x0C	? 0x0D	? 0x0E	? 0x0F
? 0x10	? 0x11	? 0x12	? 0x13	? 0x14	? 0x15	? 0x16	? 0x17
? 0x18	? 0x19	? 0x1A	? 0x1B	? 0x1C	? 0x1D	? 0x1E	? 0x1F

&n is equal to 0x08

n 4 0x08				? 0x0C	? 0x0D	? 0x0E	? 0x0F
? 0x10	? 0x11	? 0x12	? 0x13	? 0x14	? 0x15	? 0x16	? 0x17
? 0x18	? 0x19	? 0x1A	? 0x1B	? 0x1C	? 0x1D	? 0x1E	? 0x1F

```
int *p = &n;
```



Pointer Syntax

```
int *p = &n;
```

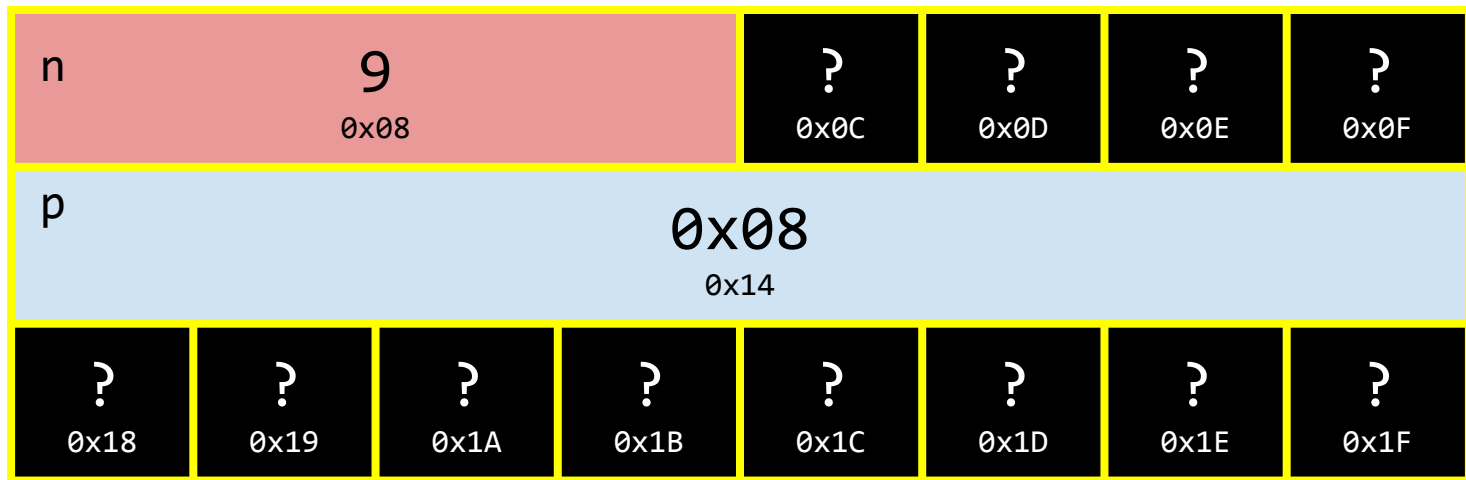
type

Pointer Syntax

```
int *p = &n;
```

"address of"

`*p = 9;`



Pointer Syntax

*p = 9;

"**dereference** (go to) the
address stored in p"

type * is a pointer that stores the address of **type**.

&x gets its address of **x**.

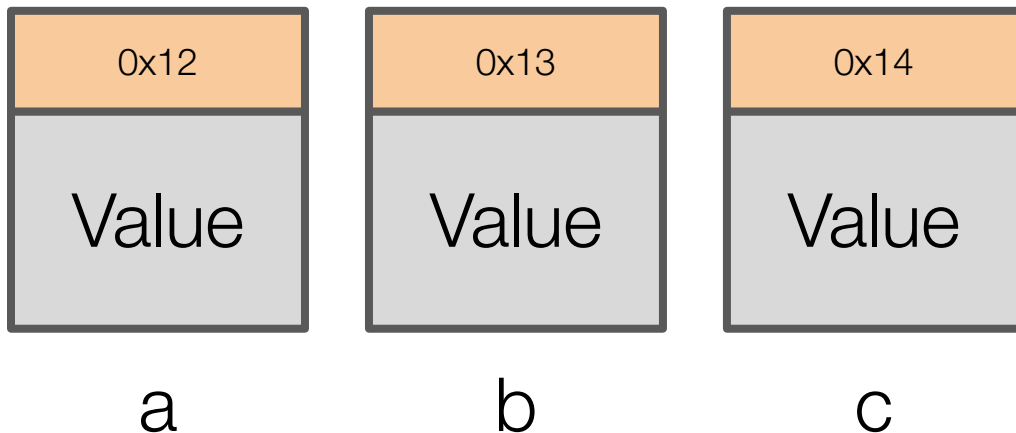
***x** goes to the address stored at the pointer **x**.

Pointer Practice

What will the final values for each variable or pointer be?

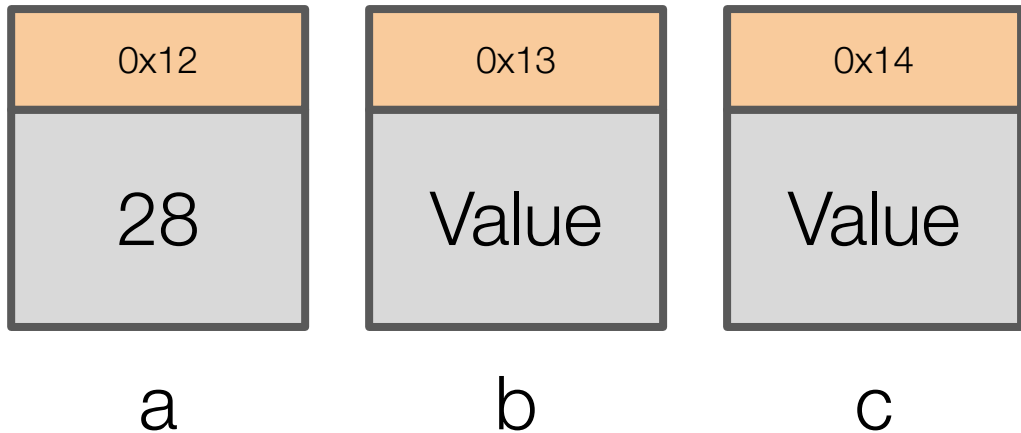
```
int a = 28;  
int b = 50;  
int *c = &a;
```

```
*c = 14;  
c = &b;  
*c = 25;
```



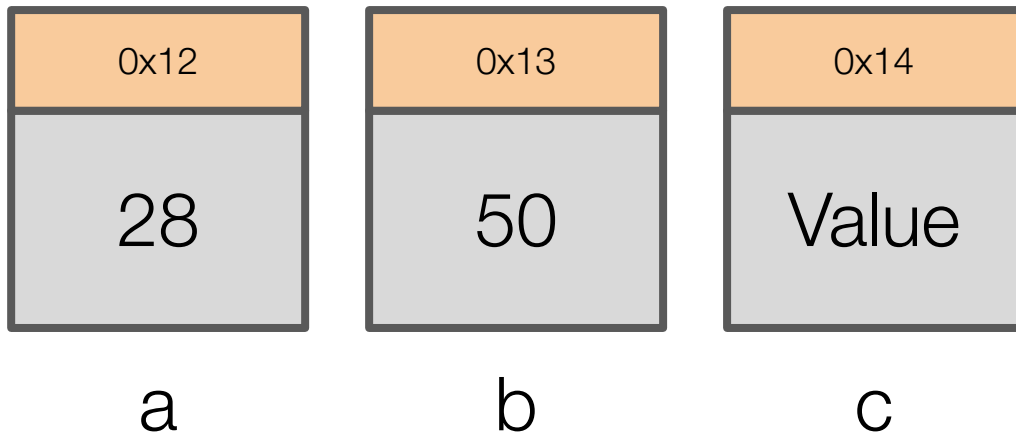
```
int a = 28;  
int b = 50;  
int *c = &a;
```

```
*c = 14;  
c = &b;  
*c = 25;
```



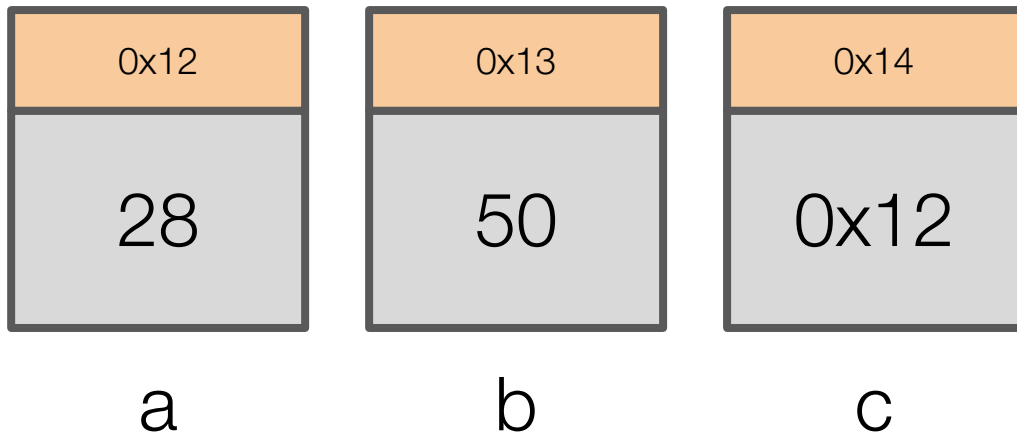
```
int a = 28;  
int b = 50;  
int *c = &a;
```

```
*c = 14;  
c = &b;  
*c = 25;
```



```
int a = 28;  
int b = 50;  
int *c = &a;
```

```
*c = 14;  
c = &b;  
*c = 25;
```

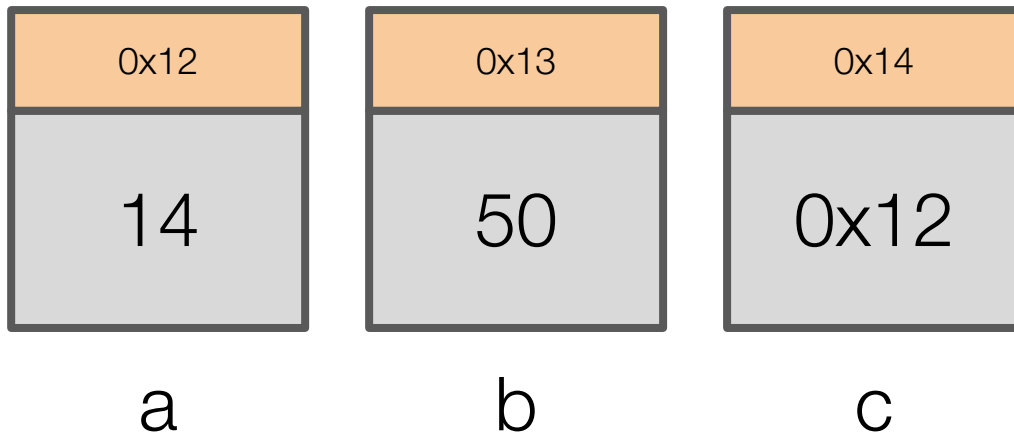


```
int a = 28;  
int b = 50;  
int *c = &a;
```

```
*c = 14;
```

```
c = &b;
```

```
*c = 25;
```

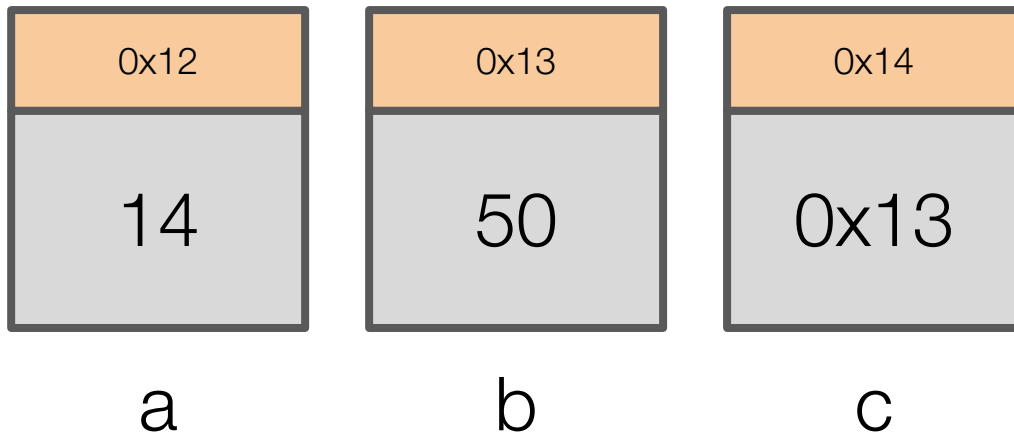


```
int a = 28;  
int b = 50;  
int *c = &a;
```

```
*c = 14;
```

```
c = &b;
```

```
*c = 25;
```

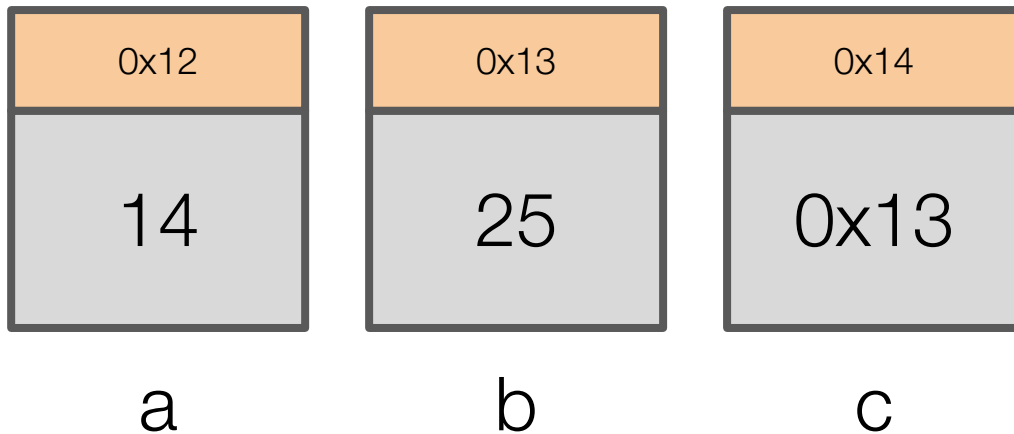


```
int a = 28;  
int b = 50;  
int *c = &a;
```

```
*c = 14;
```

```
c = &b;
```

```
*c = 25;
```

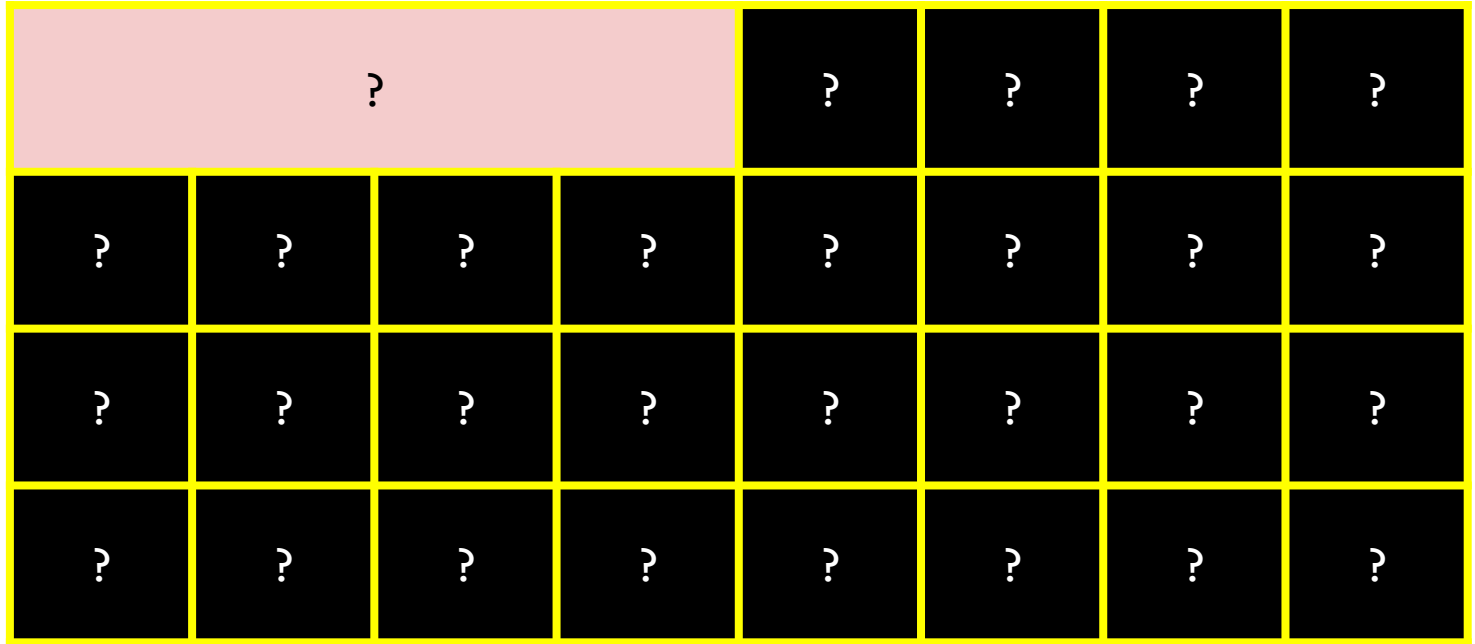


Part 2

Dynamic Memory

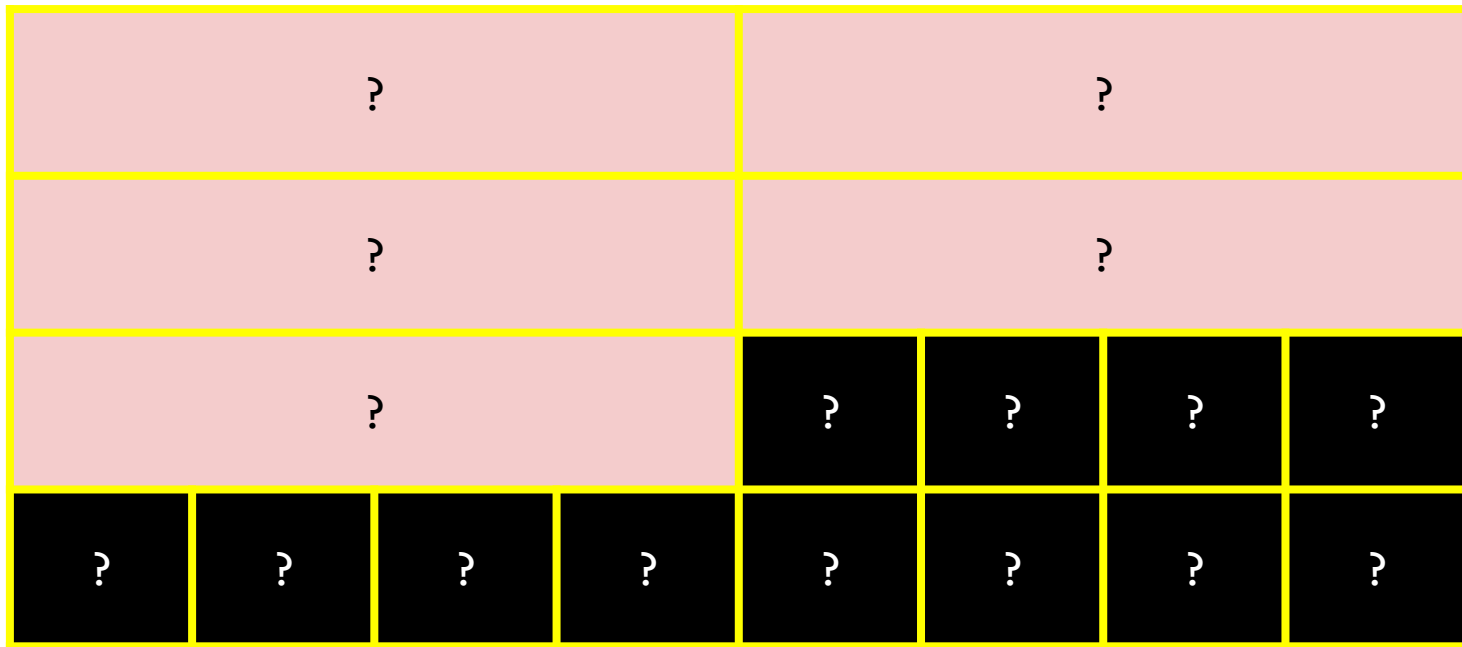
```
int *array = malloc(sizeof(int));
```

array



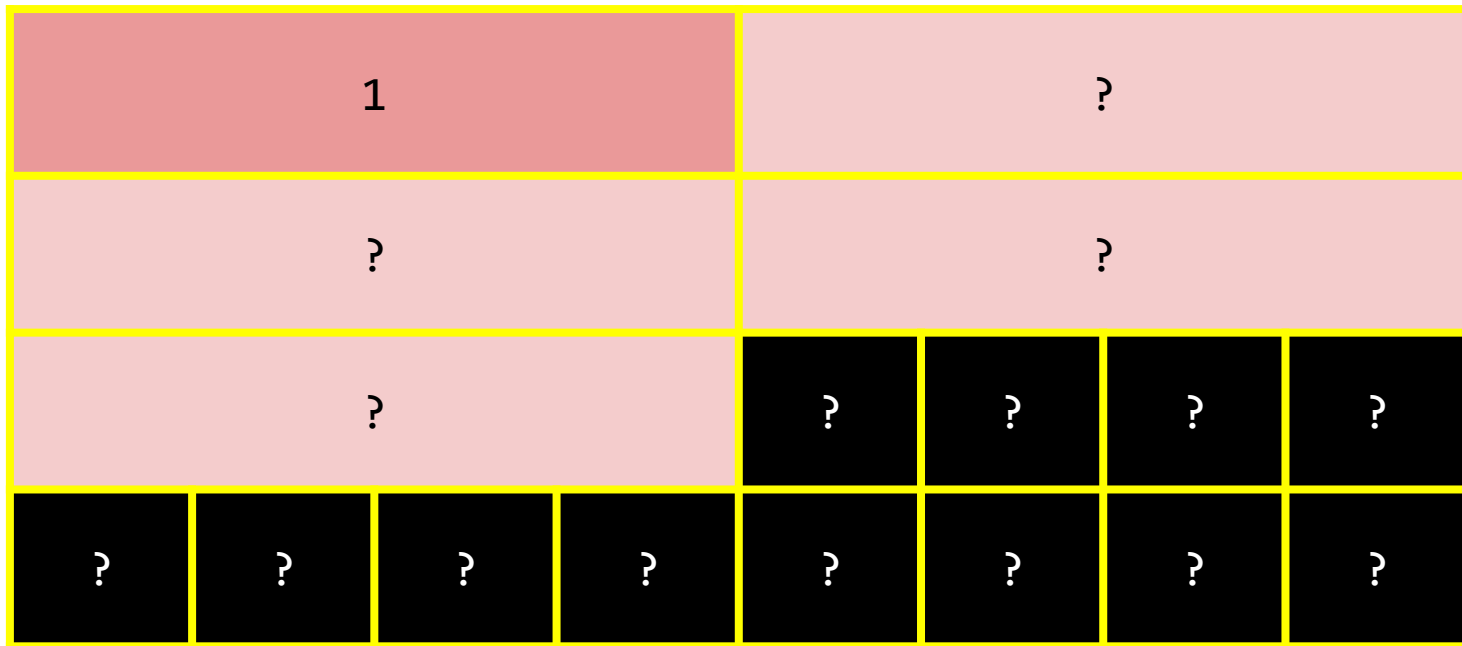
```
int *array = malloc(sizeof(int) * 5);
```

array



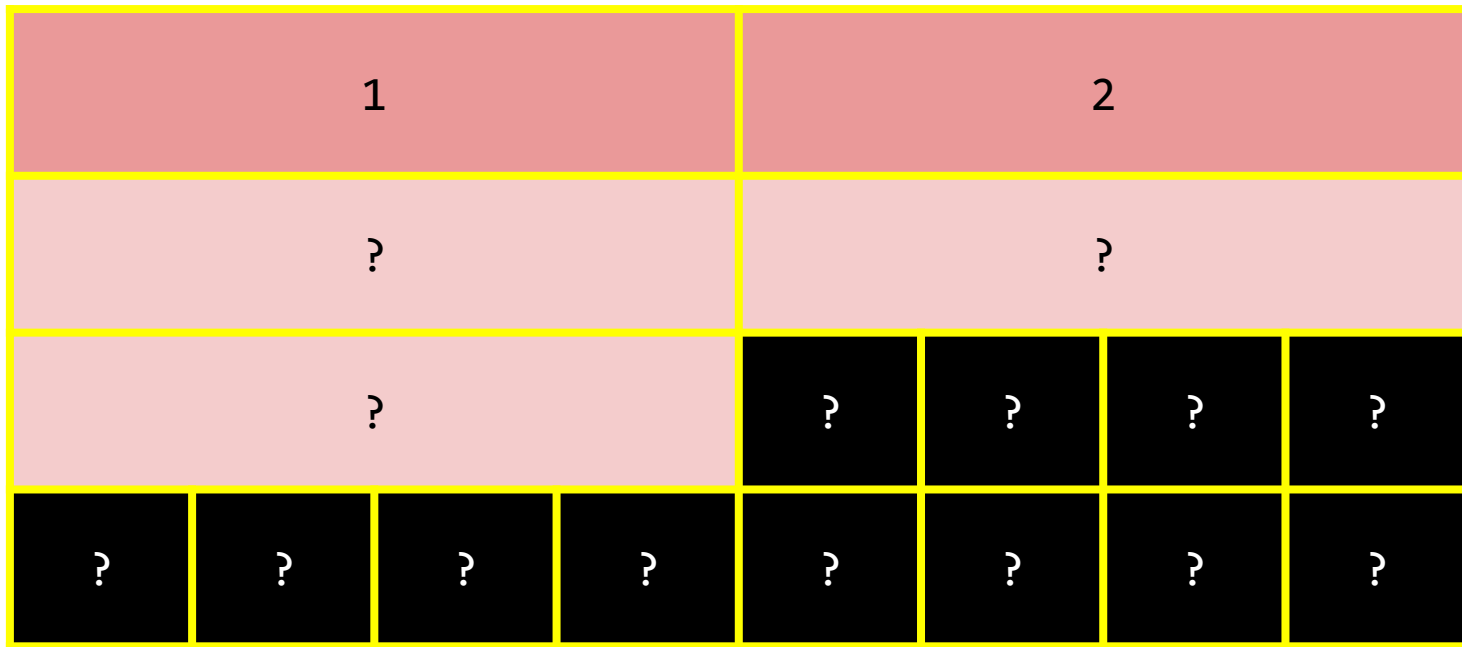
`*array = 1;`

array



```
*array = 1;  
*(array + 1) = 2;
```

array



array[2] = 3;

array



1				2			
3				?			
?				?	?	?	?
?	?	?	?	?	?	?	?

array[3] = 4;

array



1				2			
3				4			
?				?	?	?	?
?	?	?	?	?	?	?	?

```
int *array = malloc(sizeof(int) * 5);  
if (array == NULL)  
{  
    return 1;  
}
```

[...use array somehow]

```
free(array);
```

Common memory errors

- Failing to **free** every block of memory which we've **malloced**.
- Failing to **fclose** every file we've **fopened**.
- Using more memory than we've allocated.

Concatenate Exercise

Write a function, `concatenate.c`, that takes two strings and returns a concatenated version of the two strings.

Debugging Memory Exercise

Debug a program, **create.c**, that creates the file given as input at the command-line. For example,

```
./create test.c
```

will create a file, **test.c**. But our code has three memory errors! Can you find and fix them? Try running the below to check:

```
valgrind ./create test.c
```

Part 3

File I/O

How might we copy the
contents of one file into
another file?

Filename → in.txt

File contents → hi!

File Address → 0x456

out.txt

0x789

```
FILE *input = fopen("in.txt", "r");
```

in.txt



0x456

```
FILE *input = fopen("in.txt", "r");
```

name

in.txt

input



0x456

```
FILE *input = fopen("in.txt", "r");
```

type

input



in.txt



0x456

```
FILE *input = fopen("in.txt", "r");
```

value

in.txt

input

0x456

hi!

0x456

fopen Documentation

```
FILE *fopen(const char *pathname, const char *mode);
```

This function opens a file, `pathname`. Supported values for mode include

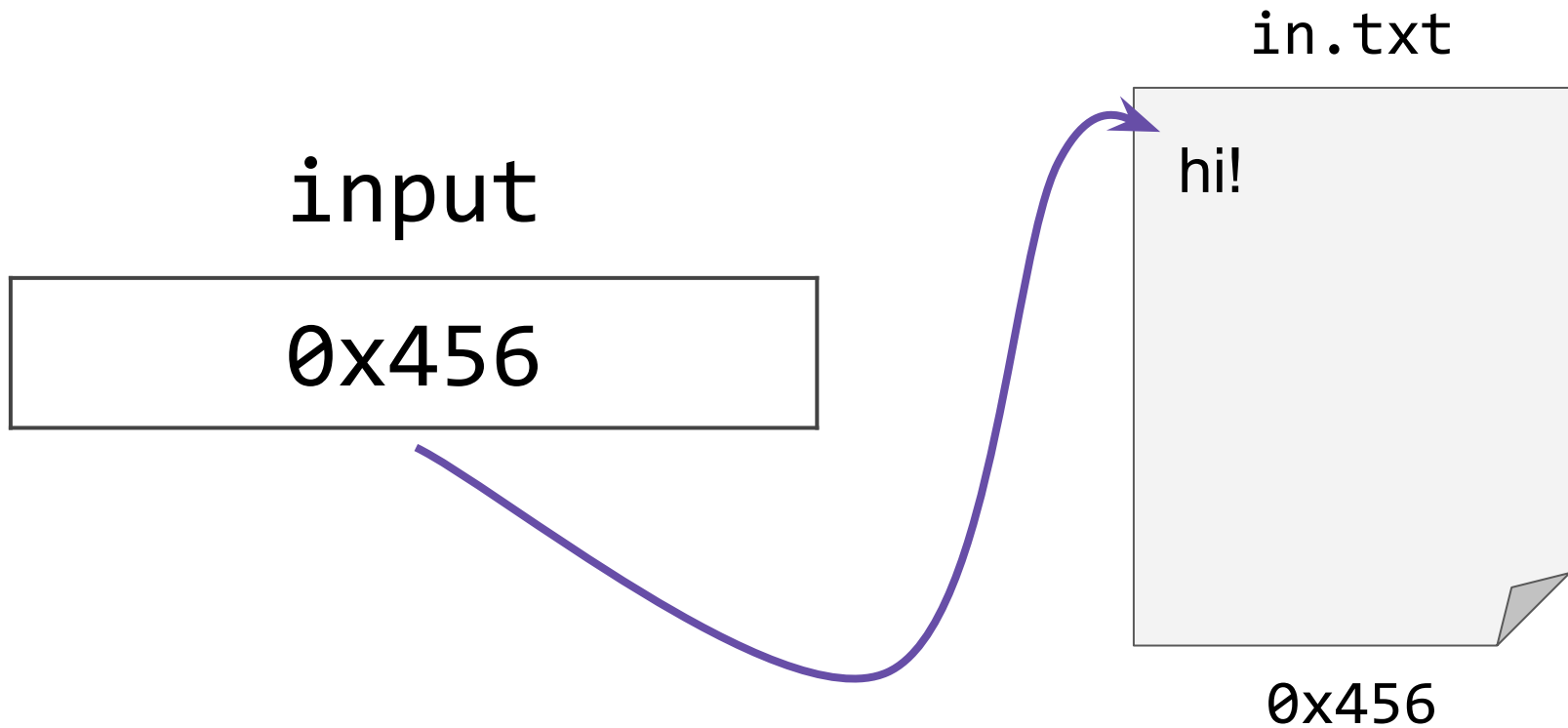
- `"r"`, if opening the file in order to read from it.
- `"w"`, if opening the file in order to write to it.
- `"a"`, if opening the file in order to append to it.

Return Value: This function returns a pointer to a `FILE` representing the opened file or, in cases of error (as when `pathname` doesn't exist), `NULL`.

Note on Strings

Recall CS50 defines `string` as an alias for `char *`.

Both represent the address of the first character in a sequence of chars, ending with `'\0'`.



in.txt

input



h

i

!

\0

```
fread(&c, sizeof(char), 1, input);
```

fread Documentation

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

This function reads data from a file that has been opened via fopen. It expects as input:

- **ptr**: address (of the first byte) of memory into which to read the data.
- **size**: size (in bytes) of the type of data to read.
- **nmemb**: number of those types to read at once.
- **stream**: pointer to a FILE returned by fopen.

Return Value: This function returns the **number of items read**. If an error occurs, or the end of the file is reached, this function might return a value smaller than **nmemb** or even 0.

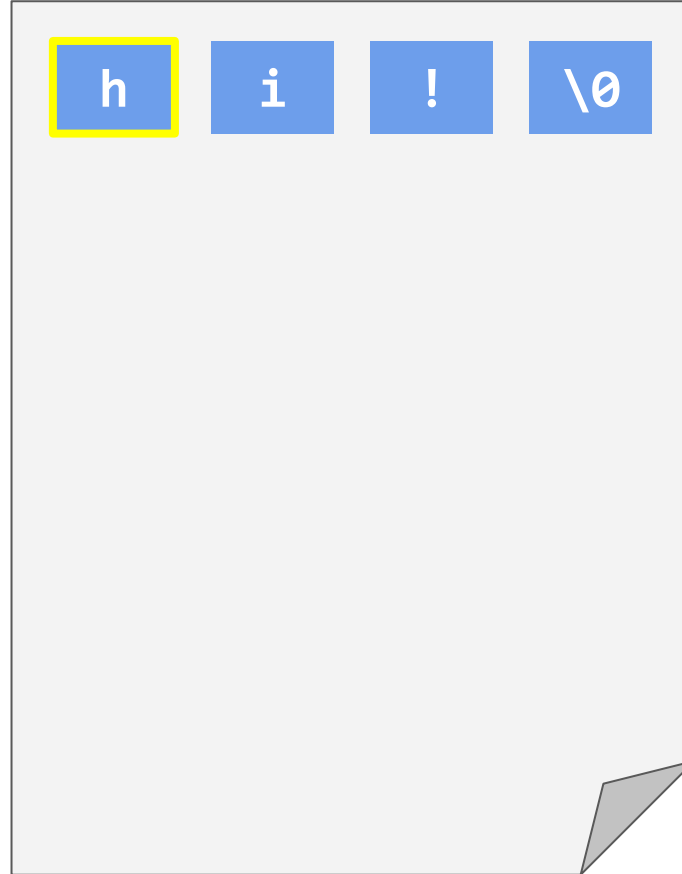
```
fread(&c, sizeof(char), 1, input);
```



Location to read from

in.txt

input



```
fread(&c, sizeof(char), 1, input);
```



Size of blocks to read (in bytes)

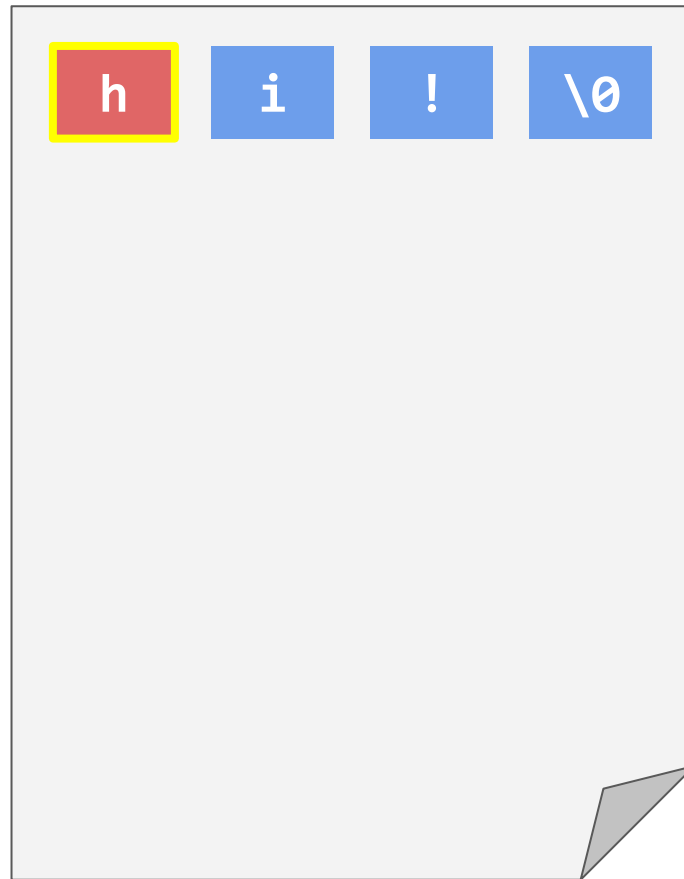
```
fread(&c, sizeof(char), 1, input);
```



How many blocks to read

in.txt

input



```
fread(&c, sizeof(char), 1, input);
```



Location to store blocks

in.txt

input →

h	i	!	\0
---	---	---	----

c

'h'

```
fwrite(&c, sizeof(char), 1, output);
```

fwrite Documentation

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

This function reads data from a file that has been opened via fopen. It expects as input:

- **ptr**: address (of the first byte) of memory into which to write the data.
- **size**: size (in bytes) of the type of data to write.
- **nmemb**: number of those types to write at once.
- **stream**: pointer to a FILE returned by fopen.

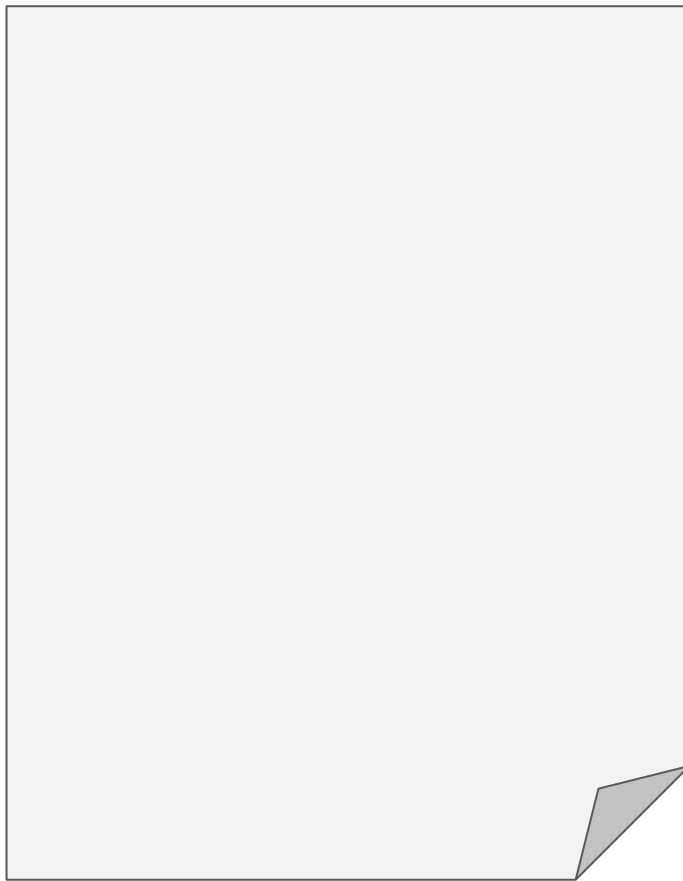
Return Value: This function returns the **number of items written**. If an error occurs, or the end of the file is reached, this function might return a value smaller than **nmemb** or even 0.

out.txt

output →

c

'h'



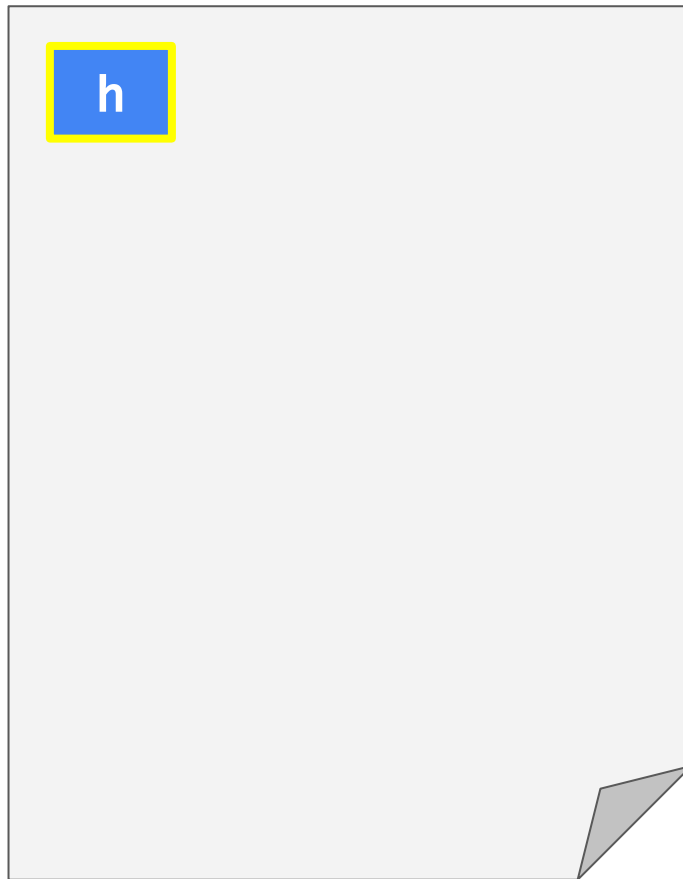
out.txt

output →

h

c

'h'



in.txt

input →

h	i	!	\0
---	---	---	----

C

'i'

out.txt

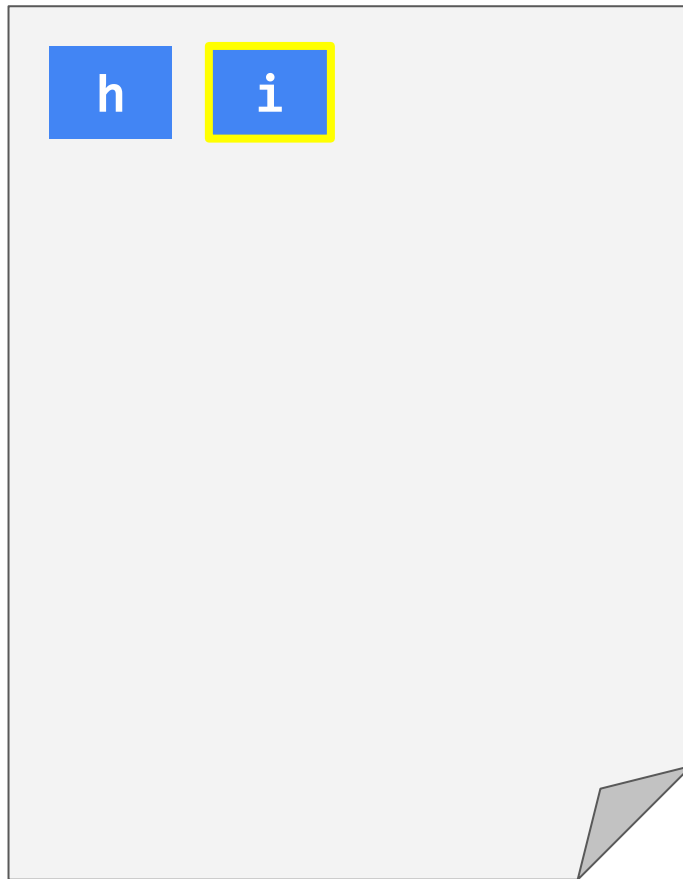
output →

h

i

C

'i'



in.txt

input →

h	i	!	\0
---	---	---	----

C

' ! '

out.txt

output



h

i

!

C

' ! '

in.txt

input →

h

i

!

\0

c

'\0'

out.txt

output



h

i

!

\0

C

'\0'

Example: copy.c

```
#include <stdio.h>
```

```
int main(void)
{
    FILE *input = fopen("in.txt", "r");
    if (input == NULL)
    {
        perror("Failed to open in.txt");
        return 1;
    }
    FILE *output = fopen("out.txt", "w");
    if (output == NULL)
    {
        perror("Failed to open out.txt");
        fclose(input);
        return 1;
    }
    char c;
    while (fread(&c, sizeof(char), 1, input) != 0)
    {
        fwrite(&c, sizeof(char), 1, output);
    }
}
```

PDF Detection Exercise

Create a program, `pdf.c`, that checks whether a file, passed in as a command-line argument, is (likely) a PDF. All PDFs will begin with a four-byte sequence:

`0x25 0x50 0x44 0x46`

```
$ ./pdf test.pdf  
Likely a PDF!  
$ ./pdf test.jpg  
Hm, not a PDF.
```

This is CS50

Week 4